

K-means Clustering

Imagine dumping a thousand jigsaw puzzle pieces onto a table with no box and no reference image. As you start grouping similar shapes and colors, patterns begin to emerge: patches of sky, edges, bits of tree or sea. Without any guidance, you slowly reconstruct structure from what initially looks like randomness. Pieces that resemble one another naturally end up together. K-Means clustering works in much the same way. It takes an unstructured collection of data points and groups them based on similarity, uncovering latent structure without being told what the “right” answer should be.

K-Means is an **unsupervised learning algorithm**, meaning it operates on data without labels. The idea of clustering using this approach was first introduced by Stuart Lloyd at Bell Labs in 1957, though it was not formally published until 1982. Since then, K-Means has become one of the most widely used clustering algorithms, with applications ranging from market segmentation and document clustering to image segmentation. In this post, I’ll walk through both the structure and the mathematics behind K-Means so you can understand not just how to use it, but how to reason about and fine-tune it.

What is K-means Clustering?

K-Means is one of the most popular clustering methods in machine learning precisely because of its simplicity and flexibility. It operates on unlabeled data, meaning the data points do not come with predefined categories. Instead of learning a mapping from inputs to outputs, K-Means groups observations based on their proximity to one another in a multidimensional feature space.

At its core, K-Means is a way of organizing chaos. You specify how many clusters you want to discover (that’s the K), and the algorithm attempts to partition the data into that many groups. Each cluster is defined by a centroid, which represents the average position of all points assigned to that cluster. Over time, these centroids shift to better reflect the structure of the data.

The dataset used in this post, Fashion-MNIST, was created by Zalando Research. We're going to use it today as a more challenging and visually meaningful alternative to the classic numerical datasets. Unlike traditional tabular datasets with rows and columns of numerical features, Fashion-MNIST consists of grayscale images. Each data point is a 28×28 pixel image representing a clothing item such as a shirt, shoe, or bag.

Each image can be thought of as a grid of 784 pixel values, which together form a high-dimensional feature vector. These pixel values define the feature space in which clustering takes place. Since K-Means is an unsupervised algorithm, the item labels are not used during training; instead, the algorithm attempts to discover natural groupings based purely on visual similarity encoded in the pixel data. Here is an example datapoint:



Figure 1

Structure

The structure of a K-Means clustering model differs fundamentally from that of supervised models like decision trees. There are no labeled outputs, no decision rules, and no hierarchical splits. Instead, the model is organized around clusters and their centroids.

At a high level, the structure consists of four components:

1. **Data points (observations):**

These are the raw input vectors. In this case, each data point corresponds to an image represented as a vector of pixel values in a continuous, multidimensional space.

2. **Centroids:**

Each cluster is summarized by a centroid, which is the mean position of all points assigned to that cluster. The centroid acts as a prototype for the group. In two dimensions, you might visualize centroids as points hovering near dense regions of data.

3. **Cluster boundaries (Voronoi regions):**

Once centroids are established, each data point is assigned to the nearest centroid using a distance metric, typically Euclidean distance. The regions formed around centroids define the cluster boundaries: every point within a region is closer to its own centroid than to any other.

4. **Assignments:**

The model maintains a mapping between data points and centroids. These assignments ultimately define the cluster structure once the algorithm converges.

The algorithm begins by randomly initializing the centroids and then iteratively refining them. In each iteration, data points are assigned to the nearest centroid, and the centroids are recomputed as the mean of their assigned points. This process repeats until the centroids stop moving significantly, indicating that the clusters have stabilized.

This alternating process consists of two steps:

- **Assignment step (E-step):** Each data point is assigned to the nearest centroid, partitioning the dataset into K clusters.
- **Update step (M-step):** Each centroid is recalculated as the mean of the points assigned to it.

A game of tug-of-war between these two steps repeats until the centroids stabilize, which means they stop moving significantly, or the assignments no longer change. This is called **convergence**, and the resulting centroids and cluster memberships together form the structure of the K-Means clustering model.

Can We Build It? Yes, We Can!

With Python, initializing and training a K-Means clustering model is remarkably straightforward – it takes just two lines of code:

```
model = KMeans(n_clusters=10, random_state=42, n_init=10)
model.fit(X, y)
```

The first line initializes the model by specifying the number of clusters to discover. In this case, we set `n_clusters=10`, which aligns with the ten clothing categories in Fashion-MNIST. The second line calls the `fit()` method, allowing the algorithm to learn the structure of the data by grouping observations based on feature similarity—in this case, similarity in pixel values.

Unlike supervised models such as XGBoost, K-Means does not learn a relationship between inputs and a target variable. There is no notion of prediction error during training because no labels are involved. Instead, the algorithm focuses entirely on organizing the data geometrically. That does not mean, however, that the algorithm is free of design choices. Even in an unsupervised setting, the behavior and stability of K-Means are shaped by a small set of parameters that control how clusters are initialized and refined.

While K-Means can be run with default settings, scikit-learn exposes several parameters that influence how the clustering behaves and how stable the results are:

1. `n_clusters`: The number of clusters the algorithm will attempt to identify. This must be chosen in advance and strongly influences the resulting structure.
2. `n_init`: The number of times the algorithm is run with different random centroid initializations. Higher values improve stability and reduce sensitivity to poor local minima.
3. `random_state`: Fixes the random seed to ensure reproducibility. Without this, different runs may produce different clusterings.
4. `max_iter`: Sets an upper bound on the number of iterations allowed before stopping, though K-Means typically converges well before this limit.

Together, these parameters do not control overfitting in the traditional supervised sense. Instead, they help ensure that the clustering is stable, interpretable, and reflective of meaningful structure in the data.

Math in the Black Box

At its core, K-Means is a geometric algorithm. It does not learn rules, weights, or decision boundaries; instead, it partitions data based on distance in feature space. When applied to the Fashion-MNIST dataset, each image becomes a point in a 784-dimensional space defined entirely by pixel values. K-Means seeks to divide this space into regions such that images within the same region are closer to one another than to images in any other region.

To accomplish this, K-Means introduces a set of K centroids, each representing the center of a cluster. The algorithm begins with an initial placement of these centroids and then iteratively refines them through two alternating steps: assigning each image to the nearest centroid, and updating each centroid to be the mean of the images assigned to it. With every iteration, this process reduces the total within-cluster distance, gradually shaping clusters that capture similarities in visual structure across the dataset.

The Objective Function

This iterative procedure is not arbitrary; it is driven by a well-defined optimization goal. K-Means formalizes the idea of “closeness” through an **objective function** that measures how compact each cluster is by minimizing the **within-cluster sum of squares (WCSS)**, also referred to as **inertia**:

$$J = \sum_{i=1}^K \sum_{x_j \in C_i} \|x_j - \mu_i\|^2$$

Where:

- x_j is a data point,
- C_i is the set of points assigned to cluster i ,
- μ_i is the centroid of cluster i ,

- and $\| \cdot \|^2$ denotes squared Euclidean distance.

This objective function captures a simple intuition: good clusters are compact clusters. By minimizing the squared distance between points and their assigned centroids, K-Means encourages data points within the same cluster to be as similar to one another as possible.

Iterative Optimization: Assignment and Update

While the objective function defines *what* K-Means is trying to minimize, it does not provide a closed-form solution for all centroids simultaneously. Instead, the algorithm optimizes this objective through an alternating, iterative process that guarantees a reduction in the loss at each step.

In the assignment step, centroid locations are held fixed and each data point is assigned to the cluster whose centroid is closest in Euclidean distance:

$$c_j = \arg \min_i \|x_j - \mu_i\|^2$$

This step optimizes cluster memberships given the current centroid locations.

In the update step, cluster assignments are held fixed and each centroid is recomputed as the mean of all points assigned to it:

$$\mu_i = \frac{1}{|C_i|} \sum_{x_j \in C_i} x_j$$

This update is optimal for squared Euclidean distance, since the mean minimizes the sum of squared deviations within a cluster. By alternating between these two steps, K-Means performs a form of coordinate descent on the objective function. Each iteration monotonically decreases the WCSS, ensuring convergence after a finite number of steps.

Initialization and Local Minima

Although K-Means is guaranteed to converge, it is not guaranteed to find the *global* minimum of the objective function. The WCSS objective is **non-convex**, meaning it can have multiple local minima. As a result, the final clustering depends heavily on the initial placement of centroids

Poor initialization can lead to suboptimal clusters, where centroids become trapped in locally optimal but globally inferior configurations. To mitigate this, modern implementations typically use K-Means++ initialization, which spreads initial centroids across the feature space. Running the algorithm multiple times with different initializations (`n_init`) and selecting the solution with the lowest inertia further improves robustness.

Each image in Fashion-MNIST is treated as a point in high-dimensional space, and clustering emerges purely from spatial proximity in pixel space. When K-Means succeeds, it is because visually similar items (such as shoes or bags) naturally cluster together based on shared low-level structure. When it struggles, that limitation becomes visible as well: pixel-level similarity often fails to capture higher-level semantic differences between items like shirts and pullovers. Seen through this lens, K-Means is both a strong baseline and a revealing diagnostic tool, offering insight into what distance-based learning can, and can't, recover from raw image data.

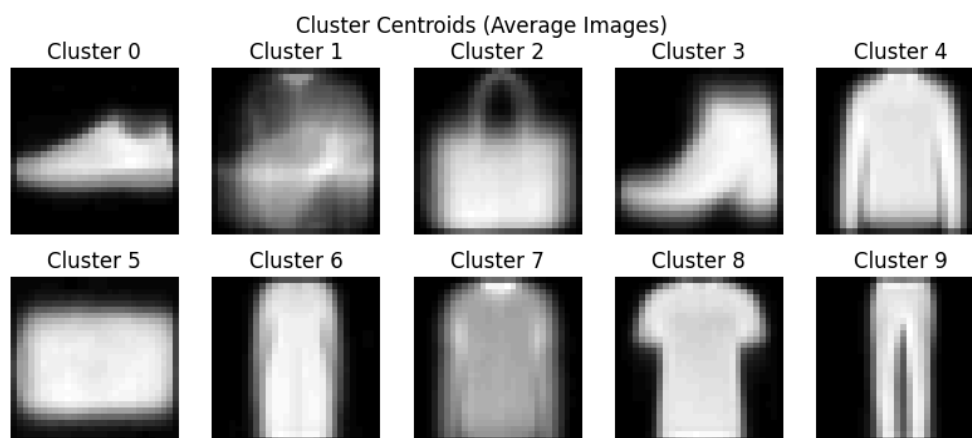


Figure 2@m

Taken together, these components describe what K-Means is optimizing and how it converges. One way to make this process more transparent is to look at the centroids themselves. Reshaping the centroids back into a 28×28 grid reveals the “average” image the algorithm has learned for that group. Figure 2 visualizes these centroids, exposing both the strengths and limitations of distance-based clustering: some clusters form clearly recognizable items, while others appear blurred or ambiguous, reflecting overlap in pixel-level structure.

In machine learning, this gap between simple interfaces and underlying mechanics is often referred to as a black box, not because the logic is unknowable, but because the implementation details are hidden behind a concise API. While scikit-learn allows us to fit a K-Means model in just a few lines of code, building the algorithm manually strips away this abstraction. Doing so makes the mechanics explicit, revealing how cluster assignments, centroid updates, and convergence arise directly from the mathematics rather than from a library call.

Computational Mathematics

We now move from theory to implementation by manually building a K-Means clustering model from scratch. The dataset is loaded from `tensorflow.keras.datasets` and lightly preprocessed to prepare the images for clustering. Rather than relying on a high-level library abstraction, we walk through each step of the pipeline explicitly—from data preparation to centroid updates. Following this process end to end is one of the most effective ways to internalize how K-Means works beneath the surface.

The `kmeans_manual` function defines a NumPy-only implementation of the K-Means algorithm. Its inputs include the data matrix X , the number of clusters k , and several parameters that control convergence and stability, such as the maximum number of iterations, a tolerance threshold, and the number of random initializations. Unlike high-level implementations, this function explicitly returns the final centroid locations, the cluster assignment for each data point, and the final value of the WCSS, making the optimization process fully transparent.

To ensure reproducibility while still allowing randomness in centroid initialization, the function creates a master random number generator seeded with a fixed `random_state`. This

generator is then used to produce independent random seeds for each initialization run. Because K-Means can converge to different local minima depending on how centroids are initialized, the algorithm is run multiple times (`n_init`), and the best solution—the one with the lowest inertia—is retained:

```
...
if inertia < best_inertia:
    best_inertia = inertia
    best_centroids = centroids
    best_labels = labels
...
```

At the start of each initialization, the algorithm selects k data points at random to serve as the initial centroids. The algorithm then enters the main optimization loop, which alternates between assignment and update steps for a fixed number of iterations or until convergence is reached.

During the assignment step, the algorithm computes the squared Euclidean distance between every data point and every centroid. Using NumPy broadcasting, this produces a distance matrix of shape (n, k) . Each data point is then assigned to the cluster corresponding to the nearest centroid via an `argmin` operation, and the current inertia is computed as the sum of squared distances to assigned centroids.

```
...
# Assignment step
distances = ((X[:, None, :] - centroids[None, :, :]) ** 2).sum(axis=2)
labels = distances.argmin(axis=1)
...
```

During the update step, new centroids are computed by taking the mean of all points assigned to each cluster. This reflects the mathematical property that the mean minimizes within-cluster squared distance. The implementation explicitly handles empty clusters by reinitializing their centroids to random data points, preventing stagnation or failure.

```
...
# Update step
new_centroids = np.zeros((k, d), dtype=X.dtype)
for ci in range(k):
    mask = labels == ci
```

```

if np.any(mask):
    new_centroids[ci] = X[mask].mean(axis=0)
else:
    new_centroids[ci] = X[rng.integers(0, n)]
...

```

After each update, convergence is checked using two criteria: the magnitude of centroid movement and the improvement in the objective function. If centroid shifts fall below a tolerance threshold or the reduction in inertia becomes negligible, the algorithm terminates early.

Once convergence is reached for a given initialization, the resulting inertia is compared to the best value observed so far. After all initializations have been evaluated, the function returns the centroids, labels, and inertia corresponding to the most compact clustering found.

By implementing K-Means in this way, the mechanics of the algorithm become explicit: distance calculations drive assignments, means drive centroid updates, and convergence emerges from monotonically decreasing within-cluster variance.

Model Output

All that remains is to prepare the data and pass it through the model. As before, the Fashion-MNIST dataset is loaded and the labels are separated, since they are not used during training. Each 28×28 image is flattened into a 784-dimensional vector and scaled to ensure consistent distance calculations. To reduce noise and improve computational efficiency, the dimensionality of the data is reduced using PCA before being passed into the manually implemented K-Means algorithm.

Once the data has been prepared, it is passed into the model. Using only pixel values, K-Means iteratively assigns each image to its nearest centroid and updates those centroids until convergence. The result is a cluster label for every image, indicating which group it belongs to based on visual similarity in feature space.

To better understand how these cluster assignments are distributed, we visualize them by projecting the dataset into two dimensions. Although clustering is performed in

higher-dimensional space, this projection provides an intuitive view of how K-Means separates data based on visual similarity.

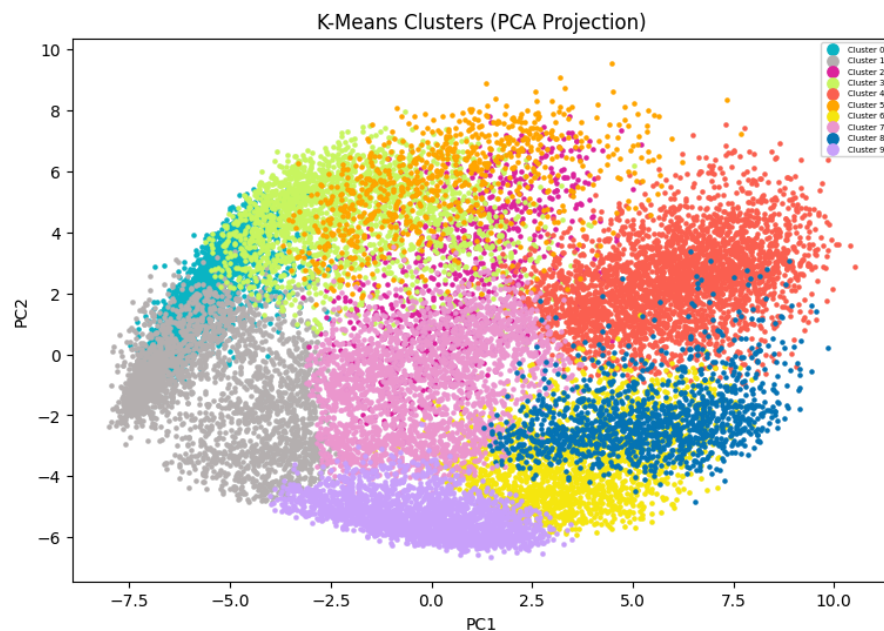


Figure 3

Figure 3 illustrates this projection. Even in two dimensions, several patterns are immediately visible. Some clusters form compact, well-separated regions, suggesting strong visual similarity, while others overlap substantially, reflecting ambiguity between certain clothing categories. This overlap is expected when clustering directly on pixel values, as visually similar items—such as shirts and pullovers—often share low-level features. While the projection does not capture the full clustering structure, it offers a useful glimpse into how K-Means partitions the data.

Visual inspection provides intuition, but it is not sufficient on its own. To more rigorously assess the quality of the clustering, we now turn to quantitative evaluation metrics.

Evaluation and Interpretation

Because K-Means is an unsupervised algorithm, evaluation focuses not on prediction error but on how well the algorithm has organized the data in feature space. In this section, we

will evaluate the learned clusters using a combination of internal and external metrics, each offering a different perspective on cluster quality and interpretability.

Beginning with internal evaluation, we assess how compact and well-separated the clusters are without relying on any labels. For this, we use the **silhouette score**, a metric designed specifically for clustering. For each data point, the silhouette score compares two quantities: the average distance to all other points in the same cluster, denoted as a , and the average distance to points in the nearest neighboring cluster, denoted as b . These are combined into the silhouette coefficient:

$$S = \frac{b-a}{\max(a,b)}$$

This score ranges from -1 to 1 , where values close to 1 indicate that a point is much closer to its own cluster than to others, values near 0 suggest overlapping clusters, and negative values indicate that a point may be assigned to the wrong cluster.

Using this approach, the clustering achieves an accuracy of 55.06% , meaning that just over half of the images are grouped into clusters that align with their true clothing categories. While this figure may appear modest compared to accuracies reported for supervised models, it is important to consider the baseline; with ten possible categories, if you were to randomly guess and assign labels, you would achieve an accuracy of around 10% . From this perspective, K-Means performs more than five times better than random assignment, despite having no access to label information during training. For additional context, a supervised model trained directly on the same representation achieves an accuracy of 55.25% . That K-Means approaches this level of performance underscores how much structure is present in the pixel space alone, even without supervision.

When ground-truth labels are available, as in the Fashion-MNIST dataset, we can also perform external evaluation to interpret the clusters semantically. Although these labels are not used during training, comparing the resulting cluster assignments to known categories helps us understand which types of images tend to be grouped together, and where the algorithm

struggles. Because cluster labels are arbitrary, we compute a **Hungarian-matched accuracy**, which finds the optimal one-to-one mapping between clusters and true labels.

Using this approach, the clustering achieves an accuracy of 55.06%, meaning that just over half of the images are grouped into clusters that align with their true clothing categories. For context, a supervised model trained directly on the same representation achieves an accuracy of 55.25%. That K-Means approaches this level of performance highlights how much structure is present in the pixel space alone, even without label information.

Deep Dive - Evaluating Clusters Without Labels

In practice, datasets like Fashion-MNIST are the exception rather than the rule. Most real-world uses of K-Means involve unlabeled data, where there is no ground truth to validate cluster assignments. For example, when clustering users of an e-commerce platform, we never truly know how “correct” a cluster is in an absolute sense.

As a result, evaluation becomes contextual rather than definitive. In production settings, the key question shifts from “*Is this cluster right?*” to “*Is this cluster useful?*” When clustering users based on browsing and purchasing behavior, clusters are evaluated by whether they exhibit distinct and interpretable patterns - differences in key metrics like conversion rate, CTR, or AOV - rather than by alignment with any predefined label.

Validation often happens indirectly through downstream impact. Clusters may be used to personalize recommendations, target campaigns, or shape on-site experiences. If introducing cluster-based logic leads to measurable improvements in the aforementioned metrics in AB tests, that improvement serves as evidence that the clustering is capturing meaningful structure. Internal metrics such as inertia and silhouette score still play a role, but primarily as sanity checks. Ultimately, in unsupervised learning, success is measured less by correctness and more by stability, interpretability, and business impact.

Taken together, these metrics provide a balanced view of the clustering results. The silhouette score reflects the inherent geometric ambiguity of pixel-based image data, while the Hungarian-matched accuracy demonstrates that the algorithm still recovers meaningful semantic groupings. This combination underscores both the limitations and the surprising effectiveness of K-Means as a baseline for unsupervised learning.

Conclusion

In this post, we stepped inside the “black box” of K-Means clustering by moving from geometric intuition to mathematical formulation and finally to a manual implementation. Rather than treating K-Means as a library call, we examined how distance-based assignments, centroid updates, and convergence emerge directly from the objective function. Applied to the Fashion-MNIST dataset, this process revealed that even simple geometric principles can uncover meaningful structure in high-dimensional image data.

The results highlight both the strengths and limitations of K-Means. Clustering directly on pixel values leads to overlapping clusters and imperfect separation, as reflected in the silhouette score. Yet, despite these constraints, the algorithm achieves a Hungarian-matched accuracy of over 55%, performing more than five times better than random assignment across ten classes. That an unsupervised model approaches the performance of a supervised baseline speaks to the surprising amount of information encoded in raw pixel space.

At the same time, this exercise makes clear why representation matters. K-Means is fundamentally limited by the features it operates on, and pixel-level distances do not fully capture the semantic relationships between images. More expressive representations—such as learned embeddings from autoencoders or convolutional neural networks—can dramatically improve clustering performance by reshaping the geometry of the feature space.

Ultimately, K-Means serves as both a practical baseline and a conceptual bridge. It illustrates how structure can emerge from data without labels, while also motivating more sophisticated approaches to unsupervised learning. Understanding its mechanics provides a solid

foundation for exploring deeper models that build on the same core idea: meaningful patterns often exist long before we tell a model what to look for.

In this post, we've learned:

- What K-Means clustering trees are
- The structure of K-Means
- The use of the objective function in K-Means clustering
- How to build a K-Means clustering model using the scikit-learn library in Python
- How to build a K-Means clustering model manually in Python
- How to predict labels using scikit and manually in Python
- How to evaluate the model using silhouette and Hungarian-matched accuracy scores
- How to interpret model evaluations
- As a bonus: How to visualize clusters in two-dimensions

That's a wrap on K-Means clustering! Next, we'll study the power of another unsupervised model - Gaussian Mixture Models. Stay tuned!